FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Millares, Kurt Martin M.
Section:	BSCSSE - AN42
Professor:	Ms. Crisola Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

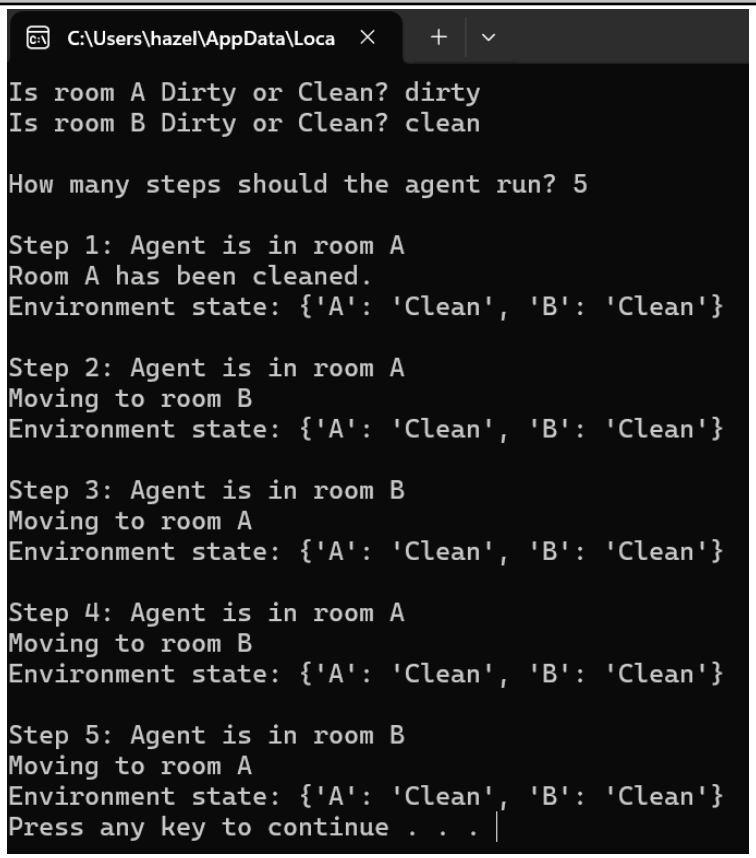
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

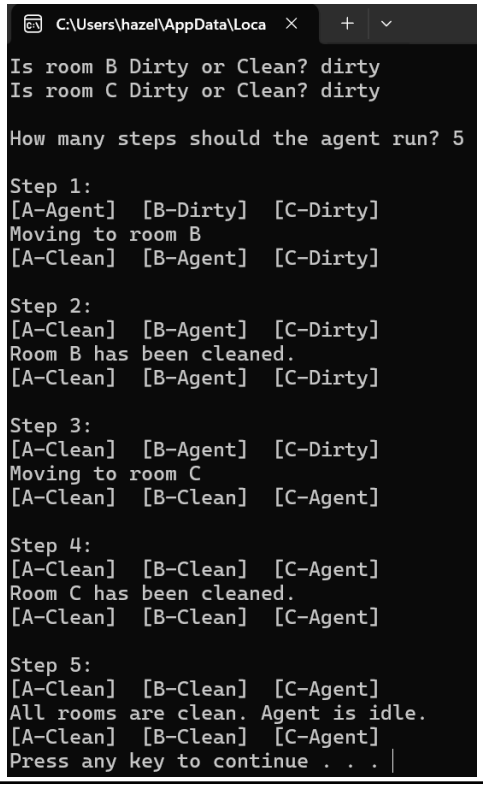
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



GitHub Repository Link:
https://github.com/kmmillares/Millares-KurtMartin-CS0065-AN42/tree/main/Technical-Assessment-2-MILLARES

Your Code - Laboratory Tasks
Task 1, 2, 3:

```
# Task 1: Set Up the Environment

# Initialize two rooms using a dictionary
environment = {
    "A": "",
    "B": ""
}


# Function to check if a room is dirty
def is_dirty(room):
    return environment[room].lower() == "dirty"


# Function to clean a room
def clean_room(room):
    environment[room] = "Clean"


# Get the initial state of each room from the user
environment["A"] = input("Is room A Dirty or Clean? ").capitalize()
environment["B"] = input("Is room B Dirty or Clean? ").capitalize()

print()


# Task 2: Implement the Rule-Based Agent

class Agent:
    def __init__(self):
        # Agent starts in room A
        self.location = "A"
```

```
    # Move to the other room
    def move(self):
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    # Perceive and act based on the rules
    def perceive_and_act(self):
        # Rule 1: If the current room is dirty, clean it
        if is_dirty(self.location):
            clean_room(self.location)
            return "Room has been cleaned."

        # Rule 2: If the current room is clean, move to the other room
        else:
            self.move()
            return f"Moving to room {self.location}"


# Task 3: Run the Simulation

# Ask the user how many steps to run
steps = int(input("How many steps should the agent run? "))

print()

# Create the agent
agent = Agent()

# Run the simulation
for step in range(1, steps + 1):

    print(f"Step {step}: Agent is in room {agent.location}")

    # Agent perceives the environment and takes an action
```

```
    action = agent.perceive_and_act()

    print(action)
    print(f"Environment state: {environment}")
    print()
```

Bonus Task:

```
import random



# ========================================
# Task 1: Set Up the Environment
# ========================================

environment = {
    "A": "",
    "B": "",
    "C": ""
}



# Check if a room is dirty
def is_dirty(room):
    return environment[room].lower() == "dirty"



# Clean a room
def clean_room(room):
    environment[room] = "Clean"



# Display the environment as a text-based grid
def display_grid(agent_location):
    rooms = []

    for room in ["A", "B", "C"]:
        if room == agent_location:
            rooms.append(f"[{room}-Agent]")
```

```
        else:
            rooms.append(f"[{room}-{environment[room]}]")

    print("    ".join(rooms))


# Get initial states
environment["A"] = input("Is room A Dirty or Clean? ").capitalize()
environment["B"] = input("Is room B Dirty or Clean? ").capitalize()
environment["C"] = input("Is room C Dirty or Clean? ").capitalize()

print()


# ========================================
# Task 2: Rule-Based Agent
# ========================================

class Agent:

    def __init__(self):
        # Agent starts in room A
        self.location = "A"

    # Move to another room
    def move(self, room):
        self.location = room

    # Perceive and act
    def perceive_and_act(self):

        # RULE 1:
        # If the current room is dirty,
        # clean it and DO NOT move.
        if is_dirty(self.location):
            clean_room(self.location)
            return f"Room {self.location} has been cleaned."
```

```
        # RULE 2:
        # If the current room is clean,
        # find the dirty rooms.
        dirty_rooms = [
            room for room in environment
            if is_dirty(room)
        ]

        # If there are dirty rooms,
        # randomly choose ONE and move to it.
        if dirty_rooms:
            next_room = random.choice(dirty_rooms)
            self.move(next_room)
            return f"Moving to room {next_room}"

        # RULE 3:
        # If all rooms are clean, stay idle.
        return "All rooms are clean. Agent is idle."


# ==========================================
# Task 3: Run the Simulation
# ==========================================

steps = int(input("How many steps should the agent run? "))

print()

agent = Agent()

for step in range(1, steps + 1):

    print(f"Step {step}:")

    # Show state BEFORE action
    display_grid(agent.location)
```

```
# Perform exactly ONE action
action = agent.perceive_and_act()

print(action)

# Show state AFTER action
display_grid(agent.location)

print()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows 2 explicit rules and 1 implicit rule. First rule, if the room it currently occupies is dirty, the agent is to clean it. Second rule, if the current room is clean, then the agent is to move to the following room. Finally, if the agent has already performed an action in the current room, for example, cleaning it or having moved into it, then it cannot execute any further processes in that same room, such as cleaning the room it moved or moving to the next room because the current one is already clean.
2. The agent decides to clean the room if it identifies it as dirty and only moves when the room is clean.
3. If both rooms were initialized to already be clean, then the agent still exhaust the allocated steps, but it will only perform moving actions.
4. For the agents to be capable of handling more than two rooms, I would simply modify the dictionary from where the rooms are stored, and allow the program to dynamically handle the number of rooms according to user input. Regardless of how many rooms there are, the agent would check the condition of the starting room, and perform either a cleaning action if it is dirty or a moving action to a randomly selected room if the current room is already clean.